Entity Relation Models

- **Entity:** An object or concept that is distinguishable from other objects. Entities can be physical objects like Students or conceptual like Courses.

- **Attributes:** Describes properties or characteristics of an entity. E.g., a Student entity may have attributes like StudentID, Name, DOB.

- **Relationship**: An association between two or more entities. For example, a Student can be enrolled in a Course (relationship between Student and Course entities).

- **Cardinality**: Defines how many instances of an entity relate to the number of instances in another entity. Common cardinalities:
  - One-to-One (1:1)
  - One-to-Many (1:N)
  - Many-to-Many (M:N)

- **ER Diagrams (ERD):** A visual representation of the entities, attributes, and relationships. Symbols:
  - **Rectangles** represent entities.
  - **Ellipses** represent attributes.
  - **Diamonds** represent relationships.
  - **Lines** connect entities to their relationships and attributes.

Normalization in a Relational Model

- **Normalization:** The process of organizing data in a database to minimize redundancy and improve data integrity.
- **Normal Forms:** Rules that a relational database must follow. Key normal forms include:
  - **First Normal Form (1NF):** Ensures that the values in a table are atomic (indivisible), i.e., each column contains unique and individual values.
  - **Second Normal Form (2NF):** Builds on 1NF by ensuring that all non-key attributes are fully functionally dependent on the primary key.
  - **Third Normal Form (3NF):** Ensures that no transitive dependencies exist, i.e., non-key attributes do not depend on other non-key attributes.

Importance of Normalization

- Prevents data anomalies like:
  - Insertion anomalies: Problems with adding new data.

- Deletion anomalies: Problems with deleting data that results in unintended data loss.
- Update anomalies: Changes in data cause inconsistencies.
- Reduces storage space
- Improves query performance

Below is an example demonstrating how a database can be normalized:

Scenario:

We have a table that stores information about students, courses, and their respective instructors. Initially, the table might look like this:

Unnormalized Table:

StudentID	StudentName	CourseID	CourseName	InstructorName
101	Alice	C101	Math	Mr. Johnson
101	Alice	C102	Physics	Dr. Lee
102	Bob	C101	Math	Mr. Johnson
103	Carol	C103	Chemistry	Dr. Smith
103	Carol	C102	Physics	Dr. Lee

Step 1: First Normal Form (1NF)

1NF requires that each table cell contains atomic (indivisible) values and that there are no repeating groups. In our table, the data is already atomic, so we are in 1NF. However, the table still has redundancies (like repeating instructor names and course names).

1NF Table:

StudentID	StudentName	CourseID	CourseName	InstructorName
101	Alice	C101	Math	Mr. Johnson
101	Alice	C102	Physics	Dr. Lee
102	Bob	C101	Math	Mr. Johnson
103	Carol	C103	Chemistry	Dr. Smith
103	Carol	C102	Physics	Dr. Lee

Step 2: Second Normal Form (2NF)

2NF eliminates partial dependencies. To achieve this, we need to ensure that non-key attributes depend on the whole primary key, not just part of it. Here, StudentName, CourseName, and InstructorName depend only on part of the primary key (CourseID or StudentID), so we break the table into two tables: one for students and their courses, and another for courses and instructors.

2NF Tables:

Students Table:

StudentID	StudentName
101	Alice
102	Bob
103	Carol

StudentCourses Table:

StudentID	CourseID
101	C101
101	C102
102	C101
103	C103
103	C102

Courses Table:

CourseID	CourseName	InstructorName
C101	Math	Mr. Johnson
C102	Physics	Dr. Lee
C103	Chemistry	Dr. Smith

Step 3: Third Normal Form (3NF)

In 3NF, we remove transitive dependencies, where non-key attributes depend on other non-key attributes. In this case, InstructorName depends on CourseID, not on the primary key. So, we split the Courses table to eliminate this dependency.

3NF Tables:

Students Table:

StudentID	StudentName
101	Alice
102	Bob
103	Carol

StudentCourses Table:

StudentID	CourseID
101	C101
101	C102
102	C101
103	C103
103	C102

Courses Table:

CourseID	CourseName
C101	Math
C102	Physics
C103	Chemistry

Instructors Table:

CourseID	InstructorName
C101	Mr. Johnson
C102	Dr. Lee
C103	Dr. Smith

Final Result:

The database is now normalized to 3NF, with all data dependencies properly organized. This removes redundancy (e.g., instructor names only appear once per course) and ensures data integrity.

- **Students Table** contains student details.
- **StudentCourses Table** links students to courses.
- **Courses Table** stores course information.
- **Instructors Table** links courses to instructors.

This structure makes it easier to update or query data without risking inconsistencies.